Computer Graphics Overview

Sure Aneesh — here's a **well-structured, detailed, and easy-to-read explanation** of **Unit 1: Computer Graphics**, rewritten from your notes so it's clear, memorable, and conceptually connected (not just a list of disjoint points).

Unit 1 – Computer Graphics

Objectives

In this unit, we aim to understand how images are created and displayed using computers. The key goals include:

- Understanding the **basic graphics system** and its working.
- Learning about **imaging systems**, both physical and synthetic.
- Studying the **synthetic camera model** and how it helps in image generation.
- Getting an idea of how programmers interact with graphics systems.
- Exploring the **design of a graphics system** and the **pipeline architecture**.
- Learning to generate simple graphics such as the **Sierpinski Gasket**.

Introduction to Computer Graphics

Computer Graphics (CG) is the field that deals with creating and manipulating visual content using computers. It encompasses everything from designing geometric models to rendering realistic scenes and animations.
At its core, CG transforms mathematical and geometrical data into visual images that humans can interpret.

Broadly, Computer Graphics involves three main activities:

1. **Geometric Modeling** – This is the process of creating mathematical representations of 2D or 3D objects.
   For example, defining a cube using its vertices, edges, and faces.
2. **Rendering** – Rendering refers to generating images from the models. It determines how light interacts with surfaces, what color each pixel should have, and creates the final picture.
3. **Animation** – Animation gives life to objects by introducing time-dependent changes, such as movement, rotation, or deformation.

A common software interface for Computer Graphics is **OpenGL (Open Graphics Library)** — a popular and powerful tool used to create both 2D and 3D graphics applications. It is easy to learn, widely supported, and follows a **top-down approach**, which means we begin by defining high-level graphics structures before specifying detailed operations.

Applications of Computer Graphics

Computer Graphics has widespread applications in almost every domain today. Let's understand the major areas:

1. Display of Information

Graphics are used as a medium to communicate complex data visually. This approach makes information more intuitive and easier to interpret.

- Historically, even ancient civilizations like the **Babylonians** displayed floor plans on stones over 4000 years ago, and **Greeks** used drawings to express architectural concepts.
- Today, graphics are used in **maps**, **charts**, **medical imaging**, and more.
    - In **medical imaging**, techniques like CT scans, MRI, and PET scans generate 3D data that can be processed and visualized using algorithms for diagnosis.
    - In **scientific visualization**, graphics help understand concepts in fluid dynamics, molecular biology, and mathematics.

---

2. Design

Interactive graphics tools are extensively used in **engineering and architecture**.
For instance:

- Architects use CG tools for **building visualization** before actual construction.
- Mechanical engineers use it for designing parts and assemblies.
- It's also essential in **VLSI design**, **animation character creation**, and **industrial design**.

---

3. Simulation and Animation

With the growth of real-time graphics systems, **simulation** became one of the most important applications.

- **Flight simulators** are a classic example — they recreate aircraft flight environments to train pilots safely and cost-effectively.
- Similarly, robotics engineers simulate robot movements and environment interactions before physical implementation.
- In the entertainment industry, animation is heavily used in **movies, games, and advertisements**, making scenes realistic and visually appealing.

A **Virtual Environment** (like VR or AR) and **Massive Multiplayer Games** are advanced forms of graphical simulations that merge real and synthetic worlds.

---

4. User Interfaces

Graphical User Interfaces (GUIs) are one of the most common uses of CG today.

- Modern computing interfaces use **windows, icons, menus, and pointers (WIMP)** to allow easy user interaction.
- The mouse, keyboard, and touch-based systems are all part of this interactive design.
- GUIs rely heavily on graphics for layout, interactivity, and feedback.

Graphics System

A **graphics system** is a computer system designed specifically for performing image generation and graphical operations.
While it has all components of a normal computer, it is optimized for graphics processing.

The five major components of a graphics system are:

1. **Input Devices**
2. **Processor**
3. **Memory**
4. **Frame Buffer**
5. **Output Devices**

---

1. Input Devices

Input devices allow users to interact with the graphics system.
The most common combination is a **keyboard** and a **pointing device** (like a mouse or joystick), which help the user specify positions or commands on the screen.
More advanced systems use **3D input devices**, such as:

- **Laser range finders** – which use laser beams to measure distances to objects.
- **Acoustic sensors** – which detect sound waves to measure spatial data or movement.

---

2. Processor

The **processor** in a graphics system handles all graphical computations.

Its main function is to take **specifications of graphical primitives** (like lines, polygons, circles) from application programs and convert them into pixel-based images.
For example, when a triangle is defined using its three vertices, the system must calculate which pixels represent its edges — this process is called **rasterization** or **scan conversion**.

Modern systems use specialized processors called **GPUs (Graphics Processing Units)** to handle these operations. GPUs are capable of executing thousands of computations simultaneously, making them ideal for rendering complex 3D scenes.

---

3. Rasterization (Scan Conversion)

Rasterization is the process of converting geometric shapes (primitives) into a grid of pixels that can be displayed on the screen.
Each pixel contains information such as **color**, **brightness**, and **depth (z-value)**.
This process forms the final **2D image** that we see on displays.

---

4. Memory

A graphics system generally contains two main types of memory:

- **General Memory:** Used for storing non-graphics data, program code, and other variables.
- **Frame Buffer:** A special memory dedicated to storing **graphics data**, i.e., pixel values representing the current image to be displayed.

---

5. Frame Buffer

The **Frame Buffer** holds the pixel data for the entire screen. Each pixel's information (such as color) is stored as bits.

Two important characteristics define a frame buffer:

- **Resolution:** The number of pixels in the buffer. For example, a resolution of 1920×1080 means the frame buffer stores over 2 million pixels.
- **Depth (Precision):** The number of bits used to represent each pixel.
  - A **1-bit** frame buffer can represent only 2 colors (black and white).
  - An **8-bit** buffer can represent 256 colors.
  - A **24-bit or 32-bit** buffer (called **True Color** or **RGB color system**) can display millions of colors and is capable of producing highly realistic images.

Understanding Pixels

An image is made up of many small square elements called **pixels (picture elements)**. Each pixel has its own color value, defined by the combination of **Red, Green, and Blue (RGB)** intensities.

When you zoom into an image, you can see these pixels as individual colored squares. The clarity or sharpness of an image depends on the **number of pixels per unit area**, known as the **resolution**.

Output Devices

The output device displays the generated image to the user. Traditionally, the **Cathode Ray Tube (CRT)** was the most common display, but it has now been largely replaced by **flat-screen displays** such as **LEDs, LCDs, and plasma screens**.

Working of a CRT

A CRT display works on the principle of electron beam excitation:

1. The processor sends a digital signal, which is converted into analog form using a **Digital-to-Analog Converter (DAC)**.
2. The analog signal controls the **X and Y deflection plates**, determining the position of the electron beam on the screen.
3. When the beam strikes the **phosphor coating** on the inside of the screen, light is emitted.
4. The continuous scanning of the beam across the screen forms the visible image.

Colored CRTs have three electron guns for red, green, and blue light. When all three beams combine, they produce different colors depending on intensity.

To prevent flicker (since the phosphor glows only briefly), the beam must refresh the screen rapidly — usually at a **refresh rate of 75–85 Hz**.

Types of CRT Displays

There are two major types of CRT systems:

Type	Description
Random Scan (Vector Display)	Also known as calligraphic display. It draws images using line segments directly between specified points. It is used for simple line drawings, such as in oscilloscopes or radar displays.
Raster Scan Display	The screen is divided into a matrix of pixels. The electron beam scans line by line from top to bottom, lighting up pixels according to frame buffer data. This is the basis for modern displays.

Raster systems are more efficient and capable of displaying complex images and animations, while random scan systems are limited but precise.

Shadow Mask CRT (Color CRT)

A **shadow mask CRT** is a technology used in color displays.
Here, three different phosphor dots (for red, green, and blue) are arranged in small triangular groups called **triads**.
Three separate electron beams, one for each color, pass through a **shadow mask** — a metal sheet with tiny holes that ensures each beam hits only its corresponding colored phosphor.
By varying the intensity of each beam, millions of colors can be produced.

Summary

Computer Graphics forms the foundation for visual computing — from simple data visualization to complex simulations and real-time 3D rendering.
The **graphics system** integrates input devices, processing units (like GPUs), memory buffers, and output displays to convert geometric data into realistic images.
Understanding concepts like **rasterization**, **frame buffers**, and **display technologies** is crucial for building efficient graphics applications and simulations.

Would you like me to continue this with the **next subtopics** — i.e., "Imaging systems: Physical and synthetic", "Synthetic camera model", "Programmer's interface", and "Graphics pipeline architecture"? I can extend in the same clear, narrative style.

Excellent — this is the continuation of **Unit 1: Computer Graphics**, covering **flat-panel displays, imaging systems, synthetic camera model, and programmer's interface (API)**.
Here's a detailed, structured, and easy-to-understand explanation rewritten in a **smooth paragraph + point blend** style for easier memorization and conceptual clarity.

Unit 1 – Computer Graphics (Continuation)

Generic Flat-Panel Display

Modern computer displays have largely shifted from bulky **CRT (Cathode Ray Tube)** monitors to **flat-panel displays**. These are much thinner, lighter, and more power-efficient, making them ideal for laptops, mobile devices, and TVs.

The most common types of flat-panel displays are:

- **LED (Light Emitting Diode)**
- **LCD (Liquid Crystal Display)**
- **Plasma Display Panels**

Structure of a Generic Flat-Panel Display

A typical flat-panel display is made up of **three main layers or plates**:

1. **Two outer plates**:
   These plates contain **parallel grids of wires** — one grid oriented horizontally and the other vertically. The crossing of these wires forms a grid pattern, and the intersection points correspond to **individual pixels**.
2. **Middle plate**:
   The central plate differs based on the type of display:
   - It can be an **LED panel**, an **LCD panel**, or a **Plasma panel**.

By sending electrical signals to specific wires in the horizontal and vertical grids, an **electric field** is generated at the intersection.
This field activates the corresponding pixel in the middle plate — turning it **on or off**, or changing its color or brightness.

Let's see how this principle differs among LED, LCD, and Plasma technologies.

---

1. LED Display

In **LED displays**, the middle panel consists of **Light Emitting Diodes**.
Each diode acts as an individual pixel that emits light when current passes through it.

- The electrical signals sent through the grid **control which LEDs are ON or OFF**, thus forming an image.
- The intensity of light can be varied to produce different brightness levels and colors.
- LEDs are known for **high brightness**, **excellent contrast**, and **low power consumption**.

There are two main types:

- **Direct (Full-Array) LED displays**, where LEDs are placed directly behind the screen.
- **Edge-Lit LED displays**, where LEDs are placed along the edges and light is distributed using diffusion layers.

---

2. LCD Display

LCD (Liquid Crystal Display) panels work differently. The middle layer is filled with **liquid crystal molecules**, which do not emit light by themselves.
Instead, they **control the amount of light** that passes through them from a backlight (often made of LEDs).

- An **electric field** is applied to change the **orientation (polarization)** of the liquid crystals.
- Depending on how the crystals are aligned, they either block or allow light to pass through each pixel.
- **Color filters** (red, green, blue) are used to form full-color images.

LCDs are widely used because they are thin, consume less power, and produce sharp images, though they typically have narrower viewing angles compared to plasma or OLED displays.

---

3. Plasma Display

Plasma panels operate on a completely different principle.
Between the glass plates of the display, there are **tiny cells filled with gas** (usually neon or xenon).
When a voltage is applied to the grids at specific intersections:

- The gas inside the cell becomes **ionized**, forming **plasma** (a glowing state of matter).
- The plasma emits **ultraviolet (UV) light**, which then excites phosphor coatings on the cell walls to produce **visible red, green, and blue light**.

Plasma displays are known for their **deep blacks**, **wide viewing angles**, and **excellent color reproduction**, although they consume more power and have largely been replaced by LED and OLED technologies.

Objects and Viewers

The next important concept in computer graphics is understanding the **relationship between objects and viewers** — because every image we see is the result of this interaction.

Objects

Objects form the **physical or geometric structure** of the scene.
In graphics, objects are represented mathematically using **geometric primitives** such as:

- **Points**
- **Lines**
- **Polygons (triangles, rectangles, etc.)**

These primitives define the **shape** and **position** of objects in a **3D coordinate space**.

For example, the following simple OpenGL code defines a **triangular polygon** using three vertices:

```
c


glBegin(GL_POLYGON);
    glVertex3f(0.0, 0.0, 0.0);
    glVertex3f(0.0, 1.0, 0.0);
    glVertex3f(1.0, 0.0, 0.0);
glEnd();
```

Here, each vertex specifies a point in 3D space (x, y, z), and the lines connecting them form the triangle.

Viewer

The **viewer** is the entity that observes or captures the image of the object.
The viewer can be:

- A **human eye**,
- A **camera**, or
- A **digitizer or sensor**.

Although the object exists in **3D space**, the viewer perceives it as a **2D image**, depending on their viewpoint.

Thus, **image formation** is the process of combining:

1. The **3D object specification**, and
2. The **viewer's parameters (position, orientation, etc.)**,

to produce a **2D projection (image)**.

Imaging System

Every imaging system (whether biological like the human eye or artificial like a camera) converts **3D real-world objects** into **2D images**.

Elements of an Imaging System

An imaging system must include the following three components:

1. **Object** – The real-world entity being viewed.
2. **Camera (or viewer)** – The device that forms the image.
3. **Light source** – Provides illumination so the object can be visible.

For image formation to occur:

- There must be a viewer or camera positioned relative to the object.
- Light must reflect from or pass through the object and reach the camera or the eye.
- The resulting 2D image is formed on a **film plane**, **sensor array**, or **retina** (in case of human vision).

In essence, both the **object and viewer** exist in a 3D world, but the **image** exists on a 2D plane.

The Programmer's Interface (API)

When developing graphical applications, programmers interact with the graphics system through a **Programmer's Interface (PI)**, also known as an **Application Programming Interface (API)**.

Purpose of the API

An API acts as a bridge between the **application program** and the **graphics hardware**.
It provides a collection of predefined **functions, classes, and commands** that allow programmers to create and manipulate graphics without directly handling hardware-level operations.

For example, in OpenGL:

- You can draw shapes, define materials, set lighting conditions, and move the camera — all using function calls.

Working of the Programmer's Interface

1. The **application programmer** writes a program using the API's functions (like OpenGL commands).
2. The API then sends this information to the **graphics drivers**.
3. The drivers translate the data into instructions that the **GPU (Graphics Processing Unit)** or hardware can understand.

This flow ensures that the programmer can focus on **graphics logic** rather than the complexity of hardware control.

For example, in a **paint program**, a user can draw shapes using menu options or mouse clicks. Even though the user isn't coding, the software internally uses API functions to render those shapes.

Three-Dimensional APIs and the Synthetic Camera Model

Most modern 3D graphics APIs such as **OpenGL**, **Direct3D**, and **OpenSceneGraph** are based on a concept called the **Synthetic Camera Model**.

This model simulates how a **real camera** captures images, allowing developers to control the 3D scene using parameters like object position, camera angle, lighting, and material properties.

The API provides functions to define:

- **Objects** (geometry and structure)
- **Viewers (camera positions and orientation)**
- **Light sources**
- **Material properties** (surface color, reflectivity, etc.)

Objects in OpenGL

In OpenGL, all **objects** are defined as collections of **vertices**.
For example, a rectangle can be represented by four vertices connected together, while a triangle requires three.

```
c


glBegin(GL_POLYGON);
    glVertex3f(0.0, 0.0, 0.0);  // vertex A
    glVertex3f(0.0, 1.0, 0.0);  // vertex B
    glVertex3f(0.0, 0.0, 1.0);  // vertex C
glEnd();
```

This specifies a **triangular polygon** in 3D space. OpenGL handles how this polygon should appear when projected onto a 2D screen.

Viewer or Camera in 3D Graphics

In 3D graphics, the **camera (or viewer)** defines how the scene is observed.
The camera can be described using four key parameters:

1. **Position** – The 3D coordinates of the camera's location, usually represented by the center of the lens or the center of projection.
2. **Orientation** – The rotation of the camera around its axes, determining which direction it's facing.
3. **Focal Length** – Determines how much of the scene the camera captures; it's similar to zooming in or out.
4. **Film Plane** – Represents the area (height and width) where the image will be projected.

Viewer Specification in OpenGL

OpenGL provides built-in functions to define the camera's properties:

1. **`gluLookAt()`** – Specifies the camera's position and orientation.

```
c

gluLookAt(eyex, eyey, eyez, centerx, centery, centerz, upx, upy, upz);
```

- **`(eyex, eyey, eyez)`** – Position of the eye or camera (where you are).
- **`(centerx, centery, centerz)`** – The point you are looking at.
- **`(upx, upy, upz)`** – Defines the "up" direction for the camera.

If not specified, OpenGL assumes:

- The camera is at the origin **`(0, 0, 0)`**
- Looking down the **negative Z-axis**
- The **Y-axis** is "up".

2. **`gluPerspective()`** – Defines the lens properties of the camera.

```
c

gluPerspective(field_of_view, aspect_ratio, near, far);
```

This determines how wide the field of view is, the proportions of the image (aspect ratio), and how far the camera can "see" into the scene.

Light Sources and Material Properties

Light Sources

Light sources in computer graphics are defined by parameters such as:

- **Location** (position in space),
- **Intensity or strength**,
- **Color**, and
- **Direction** (for directional lights).

APIs provide functions to define these parameters and control how light interacts with objects, enabling effects like shadows, highlights, and reflections.

Material Properties

Each object also has **material properties**, which determine how it responds to light.
These properties include:

- **Color**
- **Shininess**
- **Reflectivity**
- **Transparency**

In OpenGL, material attributes are usually set using functions before rendering the object. For example, setting an object's surface color or how much it reflects light.

Together, **light source settings** and **material properties** simulate realistic lighting effects, such as glossy metal, matte surfaces, or transparent glass.

Summary

To summarize, this part of the unit explains how modern displays and imaging systems work and how programmers interact with 3D graphics environments.

- **Flat-panel displays (LED, LCD, Plasma)** use different physical principles to control light emission or polarization.
- **Objects and viewers** define the structure of 3D scenes and how they're perceived as 2D images.
- The **imaging system** combines objects, light, and cameras to produce images.
- **APIs like OpenGL** allow programmers to model this process using functions for defining objects, viewers, lights, and materials — all based on the **synthetic camera model**.

Would you like me to continue the explanation for the **graphics pipeline architecture** and **generation of the Sierpinski Gasket**, which are the last parts of Unit 1? I can write them next in the same conceptual + readable format.

Unit 1 — Graphics Architectures (detailed, easy-to-remember explanation)

Below is a smooth, connected explanation that preserves every important point from your notes but arranges them into readable paragraphs mixed with short bullet reminders where helpful. I won't skip any item — you'll get Early Graphics Architectures, Display Processor Architectures, Pipeline Architectures (with each pipeline stage), the Sierpinski gasket construction and its OpenGL implementation, and the OpenGL types/primitives details.

Why graphics architectures matter

Graphics architectures describe how hardware and software are organized to convert mathematical descriptions of shapes into pictures on a screen. Over time architectures evolved to meet demands for speed, interactivity and realistic 3D imagery. The evolution went from simple, host-driven schemes to specialized display processors and finally to highly parallel pipeline hardware (today's GPUs). Each generation solved limitations of the previous one: early systems were cheap and simple but slow; display-processor designs freed the host but couldn't handle modern 3D real-time workloads; pipeline architectures provide the parallelism and VLSI integration required for high throughput and real-time 3D rendering.

---

1. Early Graphics Architectures (EGA)

Early graphics systems were built on conventional Von Neumann computers and used a single processing unit that executed instructions sequentially — one instruction at a time. The display technology used in many of these systems was the calligraphic or vector CRT: instead of refreshing a raster of pixels, the display drew lines directly by steering an electron beam along the segments that defined shapes.

In such a setup, the **host computer** had two key responsibilities: first, to compute the end points and mathematical descriptions of the primitives (for example, the endpoints of a line); second, to supply the display unit with the data fast enough to avoid flicker. Because the host had to continuously re-send drawing commands at a refresh rate, the host spent a lot of CPU time on display maintenance.

The advantages were obvious: the architecture is simple and inexpensive. The disadvantages were equally clear: it's slow and unsuitable for producing complex images or real-time 3D rendering, which led to its replacement by display-processor architectures.

---

2. Display Processor Architecture (DPA)

To reduce the host burden, systems introduced a dedicated **display processor**. In addition to the host and the output device, the hardware included a **display processor** and a **display list** (or display file) — a memory structure that holds preassembled drawing instructions.

Here's the workflow: the host computes and assembles the sequence of drawing commands once and uploads them to the display processor. The display processor stores those commands in its own memory (the display list) and then executes the list repeatedly by itself to refresh the screen at whatever rate is needed to avoid flicker. Because the display processor independently redraws the scene, the host is freed to do other computations or I/O work.

This architecture improved efficiency over EGA because the host didn't need to re-send instructions continuously. However, DPA struggled with the computational load required by real-time 3D graphics (transformations, lighting, texturing, etc.), so the industry moved toward pipeline-based solutions that exploit parallelism inside specialized VLSI chips.

---

3. Pipeline Architecture

Pipeline architecture brings the idea of **overlapping computation** to graphics. Think of the classic arithmetic pipelining example: to compute `a + (b * c)` for many triples `(a,b,c)`, a pipelined system lets the multiplier start working on a new triple while the adder finishes a previous result. The net result is increased throughput — more results per second.

Modern graphics hardware applies the same principle with many stages chained in sequence. The hardware is implemented with VLSI circuits and massively parallel units (GPUs). The typical graphics pipeline is organized into stages that transform geometry into pixels. The major steps are:

- **Vertex processing**
- **Clipping and primitive assembly**
- **Rasterization (scan conversion)**
- **Fragment processing**

Each of these stages has a specific role, and together they produce the final pixels written to the frame buffer.

(a) Vertex processing

Vertex processing is the first major stage. It treats each vertex independently and performs two main functions:

1. **Coordinate transformations** — vertices are transformed from model space to world space to camera (view) space and finally into clip/normalized device coordinates by applying matrices (model, view, projection). Because successive transformations are linear (or affine), they can be concatenated into a single transformation matrix and applied via matrix multiplication. In practice graphics APIs multiply the vertex coordinate by the combined matrix to compute its transformed position.
2. **Vertex attribute computation** — per-vertex attributes (color, normals for lighting, texture coordinates) are computed here or prepared for subsequent stages. Lighting computations that are done per-vertex (Gouraud shading) will produce vertex colors which are later interpolated across primitives.

The output of vertex processing is a set of transformed vertices with associated attributes ready to be assembled into geometric primitives.

(b) Clipping and Primitive Assembly

After vertices are processed, they must be grouped into **primitives** (lines, triangles, polygons) and any geometry outside the viewable volume must be removed or clipped. Clipping emulates the real-world camera's limited view: just as a real camera only records the scene within its view frustum, the synthetic camera defines a volume of space (the view frustum) — everything outside it will not appear in the image.

Primitive assembly is the step where vertices are combined into primitives (e.g., three vertices into a triangle). Once primitives are formed, clipping algorithms trim parts of primitives that lie outside the clip volume; the clipped primitives that remain are the ones whose projections can potentially show up in the final image.

(c) Rasterization

Rasterization (or scan conversion) converts the 2D projected primitives into discrete image samples — pixels. Each primitive is processed to generate **fragments**, where a fragment represents a potential pixel and carries information such as its screen location, interpolated color, texture coordinates, and depth value. The rasterizer's job is to determine which pixels a primitive covers and to compute those fragments for subsequent processing.

In essence, rasterization bridges continuous geometry and discrete images: primitives → fragments → potential pixels.

(d) Fragment processing

Fragment processing is where the final color of each pixel is determined. Fragments may be shaded (per-pixel lighting), have textures applied (texture mapping), and be tested against depth (z-buffer) and alpha rules. Texture mapping is specifically the process of mapping a bitmap (the texture) onto the

surface of a primitive, giving visual details that would be expensive to model geometrically. Fragment processing can also include blending, stencil tests, and other per-sample operations. Only after fragment tests pass does the fragment's color get written into the frame buffer as a pixel.

Texture mapping (brief context)

Texture mapping attaches a 2D raster image (texture) onto a 3D surface. During rasterization/fragment processing, the fragment's interpolated texture coordinates are used to sample the texture bitmap, which modulates the fragment color. This lets you add rich surface detail without increasing geometric complexity.

Construction of the Sierpinski Gasket (chaos game / random method)

The Sierpinski gasket (triangle) can be generated by a simple stochastic algorithm often called the "chaos game." The method is elegant and beautifully demonstrates how a fractal emerges from a simple iterative rule:

1. Choose any initial point inside (or near) the triangle. Call it **p**.
2. Randomly pick one of the triangle's three vertices.
3. Compute the midpoint between **p** and the selected vertex.
4. Plot this midpoint.
5. Replace **p** with this midpoint.
6. Repeat from step 2 for many iterations.

Each iteration produces a point; points accumulate and the Sierpinski triangle pattern emerges rapidly. The randomness is controlled by selecting a vertex uniformly at random each step. The algorithm highlights two important programming concepts used in graphics: **random number generation** and **recursion/iteration**.

You can structure a simple program like this:

```
c


main() {
    initialize_the_system();
    for (some_number_of_points) {
        pt = generate_a_point();
        display_the_point(pt);
    }
    cleanup();
}
```

This pseudocode repeatedly generates and displays points; the plotting is done as the points are produced so you see the fractal emerge progressively.

Implementing Sierpinski Gasket using OpenGL (explanation of the provided code)

The notes give a minimal GLUT-based OpenGL example. Let me explain the important parts and how the code fits together.

Initialization and main loop

```
c

#include <GL/glut.h>

void main(int argc, char **argv) {
    glutInit(&argc, argv);
    glutDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(500, 500);
    glutInitWindowPosition(0, 0);
    glutCreateWindow("simple OpenGL example");
    glutDisplayFunc(display);
    myinit();
    glutMainLoop();
}
```

- **`glutInit`** initializes the GLUT library and processes command-line arguments.
- **`glutDisplayMode`** sets the display mode: single buffering (**`GLUT_SINGLE`**) and RGB color (**`GLUT_RGB`**).
- **`glutCreateWindow`** opens the window.
- **`glutDisplayFunc(display)`** registers the **`display()`** function that GLUT calls whenever the window needs redrawing.
- **`glutMainLoop()`** starts the event loop; the program now waits for events and draws when requested.

The display routine with the chaos game

They define a 2D point type and a display function that iteratively builds the gasket:

```
c

typedef GLfloat point2[2];  // a 2D point type

void display(void) {
    point2 vertices[3] = { {0.0, 0.0}, {250.0, 500.0}, {500.0, 0.0} };
    static point2 p = {75.0, 50.0};  // initial point
    int j, k;

    for (k = 0; k < 5000; k++) {
        j = rand() % 3;                 // pick a random vertex 0..2
        p[0] = (p[0] + vertices[j][0]) / 2.0;
        p[1] = (p[1] + vertices[j][1]) / 2.0;

        glBegin(GL_POINTS);
            glVertex2fv(p);
        glEnd();
    }
    glFlush();  // force rendering output to appear now
}
```

- **`vertices[]`** holds the triangle corners.

- `p` is the current point; it is updated each iteration by moving halfway toward a randomly chosen vertex.
- `rand() % 3` uses the standard C random integer generator `rand()` and reduces it to `0`, `1`, or `2` with the modulus operator. This uniformly selects one of the triangle's vertices.
- Each new point is emitted as a GL point primitive with `glBegin(GL_POINTS)` and `glVertex2fv(p)`.
- `glFlush()` forces immediate drawing of buffered commands. Without it, in some environments rendering might be delayed until buffers are full or the program exits, which is not desirable when you want points to appear promptly (especially in networked or busy systems).

Note the comment in your notes: the code shown is fragmentary—the full program would also need an initialization function (`myinit`) to set up the projection and coordinate mapping, and appropriate headers and possibly seeding the random number generator (`srand`) for reproducibility or to vary the pattern.

Random number generator detail

`rand()` returns an integer pseudo-random value. Using `rand() % 3` squeezes that integer into the set {0,1,2}. Because `rand()` produces a large range, `%3` yields a near-uniform selection among the three vertices (modulo any minor bias from `rand()` implementation). If you need more control or better statistical properties, use a better RNG or techniques to avoid modulus bias; for teaching and graphics demos `rand()` is acceptable.

Basic OpenGL types and vertex functions

OpenGL defines its own basic types such as `GLfloat` and `GLint` instead of standard C types. Typically these are typedefs like `#define GLfloat float`. The reason for using API-specific types is portability: an implementation could choose different underlying types (e.g., double precision) without changing user code if the typedefs are consistent.

OpenGL provides overloaded forms of vertex commands to match your data:

- `glVertex2i(GLint x, GLint y)` — 2D integer coordinates.
- `glVertex3f(GLfloat x, GLfloat y, GLfloat z)` — 3D floating coordinates.
- `glVertex3fv(GLfloat v[3])` — pass an array pointer with three components.

This flexibility allows you to work in 2D or 3D and with either integer or floating point coordinates.

Geometric primitives in OpenGL

Primitives are specified between `glBegin()` and `glEnd()`. The argument to `glBegin` determines which primitive the listed vertices form. For example:

- `glBegin(GL_POINTS)` — each vertex is a single pixel.
- `glBegin(GL_LINES)` — pairs of vertices form independent line segments.
- `glBegin(GL_LINE_STRIP)` — connected line segments sharing vertices.

- **`glBegin(GL_TRIANGLES)`** — three vertices form a triangle (the common choice for rendering 3D models).
- **`glBegin(GL_POLYGON)`** — a general polygon (vertices define its boundary).

Example of a line segment between two 2D points:

```
c


glBegin(GL_LINES);
    glVertex2f(x1, y1);
    glVertex2f(x2, y2);
glEnd();
```

Any number of vertices can be grouped; the way they are interpreted depends on the primitive type. When building programs, triangles are preferred in modern pipelines (and GPUs are optimized for triangle lists or strips) because triangles are always planar and easy to rasterize.

Final summary and study tips

- Remember the progression: **EGA** → **DPA** → **Pipeline**. Each step moved responsibility off the host and into increasingly specialized hardware, culminating in parallel VLSI pipelines (GPUs).
- The pipeline stages (vertex → clipping/assembly → rasterization → fragment) are the backbone of modern rendering. Think about how data transforms between stages: transforms and attributes at vertices, primitives assembled and clipped, converted into fragments, then shaded and written to the frame buffer.
- The Sierpinski gasket demo ties together RNG, iteration, and immediate-mode drawing (glBegin/glEnd). It's a clear, small program showing how simple rules produce complex shapes — a great example to memorize.
- Keep in mind the OpenGL idioms: specialized GL types, multiple **`glVertex`** variants, **`glBegin/glEnd`** boundaries, and utility functions like **`glFlush`** to force rendering.

If you want, I can convert the fragmentary Sierpinski code into a complete, runnable GLUT example (with **`myinit`** that sets up orthographic projection, **`srand`** seeding, and window resizing handlers), or produce annotated diagrams that map each pipeline stage to concrete API calls. Which would help you more for revision?